Agriculture AI — RAG Implementation Plan

Python 3.13 · Gemini API · Agriculture Domain

Storage: EAV Design Pattern + pgvector + Lark Rule Engine + LLM Parser/Beautifier

Style: Constants-first · Collector-style (comprehension/pipeline) code

> Stepwise plan aligned with AI Engineering RAG Chapter (Sections 1–5) plus Section 6: Lark Rule Engine wired between two Gemini LLM calls.
>
> **Two enforced coding rules throughout every module:**
>
> 1. **Constants-first** — every literal (model name, table name, SQL string, field name, default numeric value, prompt template) is a named module-level constant. No magic strings or numbers inside functions.
> 2. **Collector style** — loops that build lists or dicts are written as comprehensions, generator expressions, or `map()` / `filter()` pipelines. Imperative `for … append` accumulation is not used.
>
> **Variable-sized schema is implemented as EAV (Entity-Attribute-Value)**: each crop attribute is a distinct row in `crop_attributes`, not a JSONB blob.

Overview

```
Phase 1 → RAG Pipeline  (Indexing → pgvector Retrieval → Generation)
Phase 2 → EAV Storage   (crop_entities + crop_attributes + agriculture_chunks)
Phase 3 → Chatbot       (pgvector top-k, constants-first prompt)
Phase 4 → RAG vs FT     (decision framework)
Phase 5 → IoT Assistant (pgvector, constants-first prompt)
Phase 6 → Full Pipeline (LLM #1 → Lark → RAG → EAV → LLM #2)
```

Prerequisites

```
python3.13 -m venv agri_env && source agri_env/bin/activate

pip install google-generativeai lark psycopg2-binary pgvector \
            python-dotenv pytest

export GEMINI_API_KEY="your_gemini_api_key_here"
export POSTGRES_DSN="postgresql://user:pass@localhost:5432/agridb"
```

Phase 1 — RAG Pipeline (Section 1)

Step 1.1 — Indexing

Every literal is a constant. Document loading, chunking, and the row-building loop are all written in collector style.

```python
# phase1_pipeline/indexing.py

import os
from pathlib import Path
from typing import Any

import psycopg2
import google.generativeai as genai
from pgvector.psycopg2 import register_vector
from dotenv import load_dotenv

load_dotenv()

# ── Constants ──────────────────────────────────────────────────────────
GEMINI_API_KEY   = os.getenv("GEMINI_API_KEY")
POSTGRES_DSN     = os.getenv("POSTGRES_DSN")
EMBED_MODEL      = "models/text-embedding-004"
TASK_DOCUMENT    = "retrieval_document"
TASK_QUERY       = "retrieval_query"
DOC_GLOB         = "*.txt"
ENCODING         = "utf-8"
DEFAULT_CHUNK    = 500
DEFAULT_OVERLAP  = 50
CHUNK_INDEX_KEY  = "chunk_index"
CONTENT_KEY      = "content"
SOURCE_KEY       = "source"

SQL_INSERT_CHUNK = """
    INSERT INTO agriculture_chunks
        (content, source, chunk_index, embedding)
    VALUES (%s, %s, %s, %s)
    RETURNING id
"""
SQL_INSERT_MAP = (
    "INSERT INTO chunk_entity_map (chunk_id, entity_id) "
    "VALUES (%s, %s)"
)

genai.configure(api_key=GEMINI_API_KEY)


# ── Connection ─────────────────────────────────────────────────────────
def _get_conn() -> Any:
    conn = psycopg2.connect(POSTGRES_DSN)
    register_vector(conn)
    return conn


# ── Document loading — collector style ─────────────────────────────────
def load_documents(docs_dir: str) -> list[dict]:
```

```python
    """Return a list of {source, content} dicts for every non-empty .txt file."""
    return [
        {SOURCE_KEY: p.name, CONTENT_KEY: p.read_text(encoding=ENCODING).strip()}
        for p in Path(docs_dir).glob(DOC_GLOB)
        if p.read_text(encoding=ENCODING).strip()
    ]


# ── Chunking — collector style ──────────────────────────────────────────
def chunk_document(
    text: str,
    chunk_size: int = DEFAULT_CHUNK,
    overlap: int    = DEFAULT_OVERLAP,
) -> list[str]:
    """Word-level sliding-window chunker. Returns list of text chunks."""
    words  = text.split()
    starts = range(0, len(words), chunk_size - overlap)
    return [
        " ".join(words[i : i + chunk_size])
        for i in starts
        if i < len(words)
    ]


# ── Embedding ───────────────────────────────────────────────────────────
def embed_text(text: str, task: str = TASK_DOCUMENT) -> list[float]:
    """Embed one text string. Returns list[float] accepted directly by pgvector."""
    return genai.embed_content(model=EMBED_MODEL, content=text,
                               task_type=task)["embedding"]


# ── Index one document — collector style ────────────────────────────────
def index_document(
    doc: dict,
    entity_id: int | None = None,
    chunk_size: int        = DEFAULT_CHUNK,
    overlap: int           = DEFAULT_OVERLAP,
) -> list[int]:
    """
    Chunk, embed, and INSERT every chunk into agriculture_chunks.
    If entity_id is given, also inserts chunk_entity_map rows.
    Returns list of new row ids — collector style over chunks.
    """
    chunks = chunk_document(doc[CONTENT_KEY], chunk_size, overlap)
    conn   = _get_conn()

    with conn, conn.cursor() as cur:
        chunk_ids = [
            _insert_chunk(cur, text, doc[SOURCE_KEY], idx, entity_id)
            for idx, text in enumerate(chunks)
        ]

    conn.close()
    return chunk_ids


def _insert_chunk(
    cur, text: str, source: str, idx: int, entity_id: int | None
) -> int:
    """Insert one chunk row and optionally its entity map row. Returns row id."""
    cur.execute(SQL_INSERT_CHUNK, (text, source, idx, embed_text(text)))
```

```
    cid = cur.fetchone()[0]
    if entity_id is not None:
        cur.execute(SQL_INSERT_MAP, (cid, entity_id))
    return cid


# ── Index a plain KB sentence ──────────────────────────────────────────────
def index_plain_text(text: str, source: str = "kb") -> int:
    """Embed and INSERT a single KB sentence with no entity link."""
    conn = _get_conn()
    with conn, conn.cursor() as cur:
        cur.execute(SQL_INSERT_CHUNK, (text, source, 0, embed_text(text)))
        cid = cur.fetchone()[0]
    conn.close()
    return cid
```

Step 1.2 — Retrieval via pgvector `<=>`

```
# phase1_pipeline/retrieval.py

import os
from typing import Any

import psycopg2
from psycopg2.extras import RealDictCursor
import google.generativeai as genai
from pgvector.psycopg2 import register_vector
from dotenv import load_dotenv

load_dotenv()

# ── Constants ──────────────────────────────────────────────────────────────
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
POSTGRES_DSN   = os.getenv("POSTGRES_DSN")
EMBED_MODEL    = "models/text-embedding-004"
TASK_QUERY     = "retrieval_query"
DEFAULT_TOP_K  = 4

# Two SQL templates — with and without crop filter.
# query_vec is bound TWICE per query: once for the SELECT similarity expression,
# once for the ORDER BY distance expression.
SQL_TOPK_ALL = """
    SELECT
        ac.id,
        ac.content,
        ac.source,
        1 - (ac.embedding <=> %s::vector) AS cosine_similarity
    FROM  agriculture_chunks ac
    ORDER BY ac.embedding <=> %s::vector
    LIMIT %s
"""
SQL_TOPK_CROP = """
    SELECT
        ac.id,
        ac.content,
        ac.source,
        ce.crop_name,
        ce.soil_type,
```

```
        ce.rainfall,
        1 - (ac.embedding <=> %s::vector) AS cosine_similarity
    FROM  agriculture_chunks  ac
    JOIN  chunk_entity_map    cem ON cem.chunk_id  = ac.id
    JOIN  crop_entities       ce  ON ce.id         = cem.entity_id
    WHERE ce.crop_name = %s
    ORDER BY ac.embedding <=> %s::vector
    LIMIT %s
"""

genai.configure(api_key=GEMINI_API_KEY)


def _get_conn() -> Any:
    conn = psycopg2.connect(POSTGRES_DSN)
    register_vector(conn)
    return conn


def _embed_query(query: str) -> list[float]:
    return genai.embed_content(
        model=EMBED_MODEL, content=query, task_type=TASK_QUERY
    )["embedding"]


def retrieve_top_k(
    query: str,
    k: int              = DEFAULT_TOP_K,
    crop_filter: str | None = None,
) -> list[dict]:
    """
    pgvector cosine similarity top-k.
    Selects SQL template based on whether crop_filter is given.
    Returns collector-style list of result dicts.
    """
    query_vec = _embed_query(query)

    sql, params = (
        (SQL_TOPK_CROP, [query_vec, crop_filter, query_vec, k])
        if crop_filter else
        (SQL_TOPK_ALL,  [query_vec, query_vec, k])
    )

    conn = _get_conn()
    try:
        with conn.cursor(cursor_factory=RealDictCursor) as cur:
            cur.execute(sql, params)
            # collector: materialize all rows as plain dicts
            return [dict(r) for r in cur.fetchall()]
    finally:
        conn.close()


def retrieve_content_only(
    query: str,
    k: int              = DEFAULT_TOP_K,
    crop_filter: str | None = None,
) -> list[str]:
    """Collector pipeline: retrieve → extract content strings only."""
    return [r["content"] for r in retrieve_top_k(query, k=k,
```

```
                                                    crop_filter=crop_filter)]
```

Phase 2 — EAV Hybrid Storage (Section 2)

Step 2.1 — Schema DDL

```sql
-- phase2_storage/schema.sql

CREATE EXTENSION IF NOT EXISTS vector;

-- Entity: one row per (crop_name, soil_type, rainfall) combination
CREATE TABLE IF NOT EXISTS crop_entities (
    id                SERIAL        PRIMARY KEY,
    crop_name         VARCHAR(100)  NOT NULL,
    soil_type         VARCHAR(50)   NOT NULL,
    rainfall          VARCHAR(20)   NOT NULL,
    base_per_hectare  NUMERIC(8, 2),
    soil_factor       NUMERIC(4, 2),
    water_adjustment  NUMERIC(4, 2),
    created_at        TIMESTAMPTZ   DEFAULT NOW(),
    UNIQUE (crop_name, soil_type, rainfall)
);

-- Attribute + Value: one row per attribute per entity
CREATE TABLE IF NOT EXISTS crop_attributes (
    id             SERIAL        PRIMARY KEY,
    entity_id      INTEGER       NOT NULL
                                 REFERENCES crop_entities(id) ON DELETE CASCADE,
    attribute      VARCHAR(100)  NOT NULL,
    value_type     VARCHAR(20)   NOT NULL
                                 CHECK (value_type IN
                                   ('integer','float','text','boolean')),
    value_integer  INTEGER,
    value_float    NUMERIC(10, 4),
    value_text     TEXT,
    value_boolean  BOOLEAN,
    UNIQUE (entity_id, attribute)
);

-- RAG chunks: fixed columns + pgvector embedding
CREATE TABLE IF NOT EXISTS agriculture_chunks (
    id           SERIAL       PRIMARY KEY,
    content      TEXT         NOT NULL,
    source       VARCHAR(200),
    chunk_index  INTEGER      DEFAULT 0,
    created_at   TIMESTAMPTZ  DEFAULT NOW(),
    embedding    vector(768)
);

-- Join table: chunk ↔ entity
CREATE TABLE IF NOT EXISTS chunk_entity_map (
    chunk_id   INTEGER NOT NULL REFERENCES agriculture_chunks(id) ON DELETE CASCADE,
    entity_id  INTEGER NOT NULL REFERENCES crop_entities(id)      ON DELETE CASCADE,
    PRIMARY KEY (chunk_id, entity_id)
);
```

```sql
-- Indexes
CREATE INDEX IF NOT EXISTS idx_chunks_ivfflat
    ON agriculture_chunks USING ivfflat (embedding vector_cosine_ops)
    WITH (lists = 100);
CREATE INDEX IF NOT EXISTS idx_entities_crop_soil
    ON crop_entities (crop_name, soil_type);
CREATE INDEX IF NOT EXISTS idx_attrs_entity_attr
    ON crop_attributes (entity_id, attribute);
CREATE INDEX IF NOT EXISTS idx_cem_entity
    ON chunk_entity_map (entity_id);
CREATE INDEX IF NOT EXISTS idx_chunks_fts
    ON agriculture_chunks USING GIN (to_tsvector('english', content));
```

Step 2.2 — EAV Store

```python
# phase2_storage/eav_store.py

import os
from typing import Any

import psycopg2
from psycopg2.extras import RealDictCursor
from dotenv import load_dotenv

load_dotenv()

# ── Constants ──────────────────────────────────────────────────────────────
POSTGRES_DSN = os.getenv("POSTGRES_DSN")

# EAV value_type → column name mapping
VALUE_COL: dict[str, str] = {
    "integer": "value_integer",
    "float":   "value_float",
    "text":    "value_text",
    "boolean": "value_boolean",
}

# Python type → EAV value_type mapping (bool checked before int — bool is int subtype)
TYPE_MAP: tuple[tuple[type, str], ...] = (
    (bool,  "boolean"),
    (int,   "integer"),
    (float, "float"),
)

SQL_UPSERT_ENTITY = """
    INSERT INTO crop_entities
        (crop_name, soil_type, rainfall,
         base_per_hectare, soil_factor, water_adjustment)
    VALUES (%s, %s, %s, %s, %s, %s)
    ON CONFLICT (crop_name, soil_type, rainfall)
    DO UPDATE SET
        base_per_hectare = EXCLUDED.base_per_hectare,
        soil_factor      = EXCLUDED.soil_factor,
        water_adjustment = EXCLUDED.water_adjustment
    RETURNING id
"""
SQL_GET_ENTITY = """
```

```
    SELECT id, crop_name, soil_type, rainfall,
           base_per_hectare, soil_factor, water_adjustment
    FROM   crop_entities
    WHERE  crop_name = %s AND soil_type = %s AND rainfall = %s
"""
SQL_GET_ATTR = """
    SELECT value_type, value_integer, value_float, value_text, value_boolean
    FROM   crop_attributes
    WHERE  entity_id = %s AND attribute = %s
"""
SQL_GET_ATTRS = """
    SELECT attribute, value_type,
           value_integer, value_float, value_text, value_boolean
    FROM   crop_attributes
    WHERE  entity_id = %s
    ORDER BY attribute
"""
SQL_GET_ATTRS_BY_NAME = """
    SELECT DISTINCT ON (ca.attribute)
           ca.attribute, ca.value_type,
           ca.value_integer, ca.value_float, ca.value_text, ca.value_boolean
    FROM   crop_attributes ca
    JOIN   crop_entities   ce ON ce.id = ca.entity_id
    WHERE  ce.crop_name = %s
    ORDER BY ca.attribute
"""
SQL_FIND_BY_ATTR = """
    SELECT ce.id, ce.crop_name, ce.soil_type, ce.rainfall,
           ce.base_per_hectare, ce.soil_factor, ce.water_adjustment
    FROM   crop_entities   ce
    JOIN   crop_attributes ca ON ca.entity_id = ce.id
    WHERE  ca.attribute = %s
      AND  ca.{vcol}    = %s
"""
# SQL_UPSERT_ATTR is built dynamically (vcol varies) but the template is constant
SQL_UPSERT_ATTR_TMPL = """
    INSERT INTO crop_attributes
        (entity_id, attribute, value_type, {vcol})
    VALUES (%s, %s, %s, %s)
    ON CONFLICT (entity_id, attribute)
    DO UPDATE SET
        value_type    = EXCLUDED.value_type,
        value_integer = CASE WHEN EXCLUDED.value_type = 'integer'
                        THEN EXCLUDED.value_integer ELSE NULL END,
        value_float   = CASE WHEN EXCLUDED.value_type = 'float'
                        THEN EXCLUDED.value_float   ELSE NULL END,
        value_text    = CASE WHEN EXCLUDED.value_type = 'text'
                        THEN EXCLUDED.value_text    ELSE NULL END,
        value_boolean = CASE WHEN EXCLUDED.value_type = 'boolean'
                        THEN EXCLUDED.value_boolean ELSE NULL END
"""


# ── Helpers ──────────────────────────────────────────────────────────────
def _get_conn() -> Any:
    return psycopg2.connect(POSTGRES_DSN)


def _infer_type(value: Any) -> str:
    """Map a Python value to its EAV value_type string."""
    return next(
```

```
        (vtype for pytype, vtype in TYPE_MAP if isinstance(value, pytype)),
        "text",
    )


def _value_col(value_type: str) -> str:
    return VALUE_COL[value_type]


def _read_value(row: dict) -> Any:
    """Extract the typed value from one EAV result row dict."""
    return row[_value_col(row["value_type"])]


# ── EavStore ──────────────────────────────────────────────────────────
class EavStore:
    """
    EAV read/write layer for crop_entities and crop_attributes.
    All public methods are self-contained (open + close their own connection).
    """

    # ── Entity ────────────────────────────────────────────────────────

    def upsert_entity(
        self,
        crop_name: str, soil_type: str, rainfall: str,
        base_per_hectare: float | None = None,
        soil_factor:      float | None = None,
        water_adjustment: float | None = None,
    ) -> int:
        conn = _get_conn()
        try:
            with conn, conn.cursor() as cur:
                cur.execute(SQL_UPSERT_ENTITY,
                            (crop_name, soil_type, rainfall,
                             base_per_hectare, soil_factor, water_adjustment))
                return cur.fetchone()[0]
        finally:
            conn.close()

    def get_entity(self, crop_name: str, soil_type: str,
                   rainfall: str) -> dict | None:
        conn = _get_conn()
        try:
            with conn.cursor(cursor_factory=RealDictCursor) as cur:
                cur.execute(SQL_GET_ENTITY, (crop_name, soil_type, rainfall))
                row = cur.fetchone()
        finally:
            conn.close()
        return dict(row) if row else None

    # ── Attributes ────────────────────────────────────────────────────

    def set_attr(self, entity_id: int, attribute: str,
                 value: Any, value_type: str | None = None) -> None:
        vtype = value_type or _infer_type(value)
        vcol  = _value_col(vtype)
        sql   = SQL_UPSERT_ATTR_TMPL.format(vcol=vcol)
        conn  = _get_conn()
        try:
            with conn, conn.cursor() as cur:
```

```
            cur.execute(sql, (entity_id, attribute, vtype, value))
    finally:
        conn.close()

def set_attrs(self, entity_id: int,
              attrs: dict[str, tuple[Any, str]]) -> None:
    """
    Bulk-set attributes in one connection.
    attrs = {attribute: (value, value_type)} dict.
    Collector style: builds all SQL+param pairs then executes each.
    """
    conn = _get_conn()
    try:
        with conn, conn.cursor() as cur:
            # collector: generate (sql, params) for every attribute
            statements = [
                (SQL_UPSERT_ATTR_TMPL.format(vcol=_value_col(vtype)),
                 (entity_id, attr, vtype, val))
                for attr, (val, vtype) in attrs.items()
            ]
            # execute each without leaving the connection
            for sql, params in statements:
                cur.execute(sql, params)
    finally:
        conn.close()

def get_attr(self, entity_id: int, attribute: str) -> Any | None:
    conn = _get_conn()
    try:
        with conn.cursor(cursor_factory=RealDictCursor) as cur:
            cur.execute(SQL_GET_ATTR, (entity_id, attribute))
            row = cur.fetchone()
    finally:
        conn.close()
    return _read_value(dict(row)) if row else None

def get_attrs(self, entity_id: int) -> dict:
    """Collector: fetch all EAV rows → {attribute: typed_value} dict."""
    conn = _get_conn()
    try:
        with conn.cursor(cursor_factory=RealDictCursor) as cur:
            cur.execute(SQL_GET_ATTRS, (entity_id,))
            rows = cur.fetchall()
    finally:
        conn.close()
    return {row["attribute"]: _read_value(dict(row)) for row in rows}

def get_attrs_by_name(self, crop_name: str) -> dict:
    """Collector: join entities → attributes → {attribute: typed_value}."""
    conn = _get_conn()
    try:
        with conn.cursor(cursor_factory=RealDictCursor) as cur:
            cur.execute(SQL_GET_ATTRS_BY_NAME, (crop_name,))
            rows = cur.fetchall()
    finally:
        conn.close()
    return {row["attribute"]: _read_value(dict(row)) for row in rows}

def find_entities_by_attr(
    self, attribute: str, value: Any, value_type: str | None = None
) -> list[dict]:
```

```
        """Inverse EAV query: 'which crop entities have growth_days = 120?'"""
        vtype = value_type or _infer_type(value)
        vcol  = _value_col(vtype)
        sql   = SQL_FIND_BY_ATTR.format(vcol=vcol)
        conn  = _get_conn()
        try:
            with conn.cursor(cursor_factory=RealDictCursor) as cur:
                cur.execute(sql, (attribute, value))
                return [dict(r) for r in cur.fetchall()]
        finally:
            conn.close()
```

Step 2.3 — Unified PgHybridStore

```
# phase2_storage/pg_hybrid_store.py

import os
from typing import Any

import psycopg2
from psycopg2.extras import RealDictCursor
import google.generativeai as genai
from pgvector.psycopg2 import register_vector
from dotenv import load_dotenv

from .eav_store import EavStore

load_dotenv()

# ── Constants ──────────────────────────────────────────────────────────
GEMINI_API_KEY  = os.getenv("GEMINI_API_KEY")
POSTGRES_DSN    = os.getenv("POSTGRES_DSN")
EMBED_MODEL     = "models/text-embedding-004"
TASK_DOCUMENT   = "retrieval_document"
TASK_QUERY      = "retrieval_query"
DEFAULT_TOP_K   = 4
DEFAULT_SOURCE  = "kb"
DEFAULT_CHUNK_IDX = 0
RELATED_PREFIX  = "related_crop_"

SQL_INSERT_CHUNK = """
    INSERT INTO agriculture_chunks
        (content, source, chunk_index, embedding)
    VALUES (%s, %s, %s, %s) RETURNING id
"""
SQL_INSERT_MAP = (
    "INSERT INTO chunk_entity_map (chunk_id, entity_id) "
    "VALUES (%s, %s) ON CONFLICT DO NOTHING"
)
SQL_TOPK_ALL = """
    SELECT
        ac.content, ac.source,
        1 - (ac.embedding <=> %s::vector) AS cosine_similarity
    FROM  agriculture_chunks ac
    ORDER BY ac.embedding <=> %s::vector
    LIMIT %s
"""
SQL_TOPK_CROP = """
```

```
    SELECT
        ac.content, ac.source,
        ce.crop_name, ce.soil_type, ce.rainfall,
        1 - (ac.embedding <=> %s::vector) AS cosine_similarity
    FROM  agriculture_chunks ac
    JOIN  chunk_entity_map   cem ON cem.chunk_id = ac.id
    JOIN  crop_entities      ce  ON ce.id        = cem.entity_id
    WHERE ce.crop_name = %s
    ORDER BY ac.embedding <=> %s::vector
    LIMIT %s
"""
SQL_TOPK_RELATED = """
    SELECT
        ac.content, ac.source, ce.crop_name,
        1 - (ac.embedding <=> %s::vector) AS cosine_similarity
    FROM  agriculture_chunks ac
    JOIN  chunk_entity_map   cem ON cem.chunk_id = ac.id
    JOIN  crop_entities      ce  ON ce.id        = cem.entity_id
    WHERE ce.crop_name IN ({placeholders})
    ORDER BY ac.embedding <=> %s::vector
    LIMIT %s
"""

genai.configure(api_key=GEMINI_API_KEY)


def _get_conn() -> Any:
    conn = psycopg2.connect(POSTGRES_DSN)
    register_vector(conn)
    return conn


def _embed(text: str, task: str) -> list[float]:
    return genai.embed_content(model=EMBED_MODEL, content=text,
                               task_type=task)["embedding"]


class PgHybridStore:
    """
    Unified store: EAV (EavStore) + pgvector retrieval.
    All literals are constants. All result collection is comprehension-style.
    """

    def __init__(self):
        self.eav = EavStore()

    # ── Write ──────────────────────────────────────────────────────────

    def insert(self, content: str,
               source: str    = DEFAULT_SOURCE,
               chunk_index: int = DEFAULT_CHUNK_IDX) -> int:
        emb  = _embed(content, TASK_DOCUMENT)
        conn = _get_conn()
        try:
            with conn, conn.cursor() as cur:
                cur.execute(SQL_INSERT_CHUNK, (content, source, chunk_index, emb))
                return cur.fetchone()[0]
        finally:
            conn.close()

    def link_chunk_entity(self, chunk_id: int, entity_id: int) -> None:
```

```
    conn = _get_conn()
    try:
        with conn, conn.cursor() as cur:
            cur.execute(SQL_INSERT_MAP, (chunk_id, entity_id))
    finally:
        conn.close()

def bulk_insert(self, rows: list[dict],
                entity_id: int | None = None) -> list[int]:
    """
    Collector style: embed all rows then INSERT in one transaction.
    Returns list of new row ids.
    """
    conn = _get_conn()
    try:
        with conn, conn.cursor() as cur:
            ids = [
                _insert_and_link(cur, row, entity_id)
                for row in rows
            ]
    finally:
        conn.close()
    return ids

# ── pgvector retrieval ──────────────────────────────────────────────────

def retrieve_top_k(
    self,
    query: str,
    k: int              = DEFAULT_TOP_K,
    crop_filter: str | None = None,
) -> list[dict]:
    """
    Collector style: embed query → execute SQL → return list[dict].
    SQL template selected by whether crop_filter is given.
    """
    qv  = _embed(query, TASK_QUERY)
    sql, params = (
        (SQL_TOPK_CROP, [qv, crop_filter, qv, k])
        if crop_filter else
        (SQL_TOPK_ALL,  [qv, qv, k])
    )
    conn = _get_conn()
    try:
        with conn.cursor(cursor_factory=RealDictCursor) as cur:
            cur.execute(sql, params)
            return [dict(r) for r in cur.fetchall()]
    finally:
        conn.close()

def retrieve_content_only(self, query: str,
                          k: int = DEFAULT_TOP_K,
                          crop_filter: str | None = None) -> list[str]:
    """Collector pipeline: retrieve_top_k → pluck 'content' strings."""
    return [r["content"] for r in self.retrieve_top_k(query, k, crop_filter)]

def traverse_related(self, crop: str, query: str,
                     k: int = DEFAULT_TOP_K) -> list[dict]:
    """
    EAV graph hop: read related_crop_* attrs → pgvector search over those crops.
    Collector style throughout.
```

```
        """
        attrs   = self.eav.get_attrs_by_name(crop)
        # collector: filter attribute names starting with RELATED_PREFIX
        related = [
            v for key, v in attrs.items()
            if key.startswith(RELATED_PREFIX) and isinstance(v, str)
        ]
        if not related:
            return self.retrieve_top_k(query, k=k)

        qv           = _embed(query, TASK_QUERY)
        placeholders = ", ".join(["%s"] * len(related))
        sql          = SQL_TOPK_RELATED.format(placeholders=placeholders)
        params       = [qv] + related + [qv, k]

        conn = _get_conn()
        try:
            with conn.cursor(cursor_factory=RealDictCursor) as cur:
                cur.execute(sql, params)
                return [dict(r) for r in cur.fetchall()]
        finally:
            conn.close()

    # ── EAV delegations ──────────────────────────────────────────────

    def get_rule_facts(self, crop: str, soil_type: str,
                       rainfall: str) -> dict | None:
        return self.eav.get_entity(crop, soil_type, rainfall)

    def get_crop_profile(self, entity_id: int) -> dict:
        return self.eav.get_attrs(entity_id)

    def get_crop_profile_by_name(self, crop_name: str) -> dict:
        return self.eav.get_attrs_by_name(crop_name)


# ── Module-level helper used by bulk_insert ──────────────────────────
def _insert_and_link(cur, row: dict, entity_id: int | None) -> int:
    emb = _embed(row["content"], TASK_DOCUMENT)
    cur.execute(SQL_INSERT_CHUNK,
                (row["content"], row.get("source", DEFAULT_SOURCE),
                 row.get("chunk_index", DEFAULT_CHUNK_IDX), emb))
    cid = cur.fetchone()[0]
    if entity_id is not None:
        cur.execute(SQL_INSERT_MAP, (cid, entity_id))
    return cid
```

Step 2.4 — EAV Seed Data

```
# phase2_storage/seed_data.py

import os
from dotenv import load_dotenv
from phase2_storage.eav_store       import EavStore
from phase2_storage.pg_hybrid_store import PgHybridStore

load_dotenv()
```

```python
# ── Constants ──────────────────────────────────────────────────────────────
KB_SOURCE      = "agri_kb"

# Entity definitions: (crop, soil, rainfall, base_per_ha, soil_factor, water_adj)
ENTITY_SPECS: list[tuple] = [
    ("wheat",  "sandy", "low",    80.0, 1.2, 1.3),
    ("wheat",  "clay",  "medium", 80.0, 1.5, 1.1),
    ("rice",   "clay",  "high",  100.0, 1.5, 0.9),
    ("maize",  "loamy", "medium", 90.0, 1.0, 1.1),
    ("tomato", "loamy", "medium",150.0, 1.0, 1.1),
]

# EAV attributes per entity key (crop, soil, rainfall)
# Format: {attribute: (value, value_type)}
ENTITY_ATTRS: dict[tuple, dict] = {
    ("wheat", "sandy", "low"): {
        "growth_days":       (120,     "integer"),
        "irrigation_stages": (6,       "integer"),
        "pH_range_min":      (6.0,     "float"),
        "pH_range_max":      (7.5,     "float"),
        "water_req_mm":      (550.0,   "float"),
        "leaching_risk":     ("high",  "text"),
        "split_doses":       (3,       "integer"),
        "related_crop_1":    ("barley","text"),
        "related_crop_2":    ("oats",  "text"),
    },
    ("wheat", "clay", "medium"): {
        "growth_days":        (125,      "integer"),
        "irrigation_stages":  (5,        "integer"),
        "pH_range_min":       (6.5,      "float"),
        "pH_range_max":       (7.5,      "float"),
        "drainage_needed":    (True,     "boolean"),
        "waterlogging_risk":  ("medium", "text"),
        "related_crop_1":     ("barley", "text"),
    },
    ("rice", "clay", "high"): {
        "growth_days":               (110,      "integer"),
        "water_depth_cm":            (5.0,      "float"),
        "flood_weeks":               (14,       "integer"),
        "transplant_method":         ("manual", "text"),
        "drain_before_harvest_weeks":(2,        "integer"),
        "related_crop_1":            ("wheat",  "text"),
    },
    ("maize", "loamy", "medium"): {
        "growth_days":       (90,              "integer"),
        "row_spacing_cm":    (75,              "integer"),
        "hill_spacing_cm":   (25,              "integer"),
        "drought_tolerance": ("medium",        "text"),
        "critical_stage_1":  ("tasseling",     "text"),
        "critical_stage_2":  ("silk_emergence","text"),
        "related_crop_1":    ("sorghum",       "text"),
        "related_crop_2":    ("millet",        "text"),
    },
    ("tomato", "loamy", "medium"): {
        "growth_days":               (75,    "integer"),
        "staking_height_cm":         (30,    "integer"),
        "fruit_set_min_C":           (15.0,  "float"),
        "fruit_set_max_C":           (30.0,  "float"),
        "drip_liters_per_plant_day": (4.5,   "float"),
        "transplant_spacing_cm":     (60,    "integer"),
        "pruning_required":          (True,  "boolean"),
```

```
        "split_doses":               (4,     "integer"),
    },
}

# Content chunks per entity key
ENTITY_CONTENT: dict[tuple, tuple[str, str]] = {
    ("wheat", "sandy", "low"): (
        "Wheat on sandy soil, low rainfall: base 80 kg/ha, soil factor 1.2. "
        "20% extra nitrogen required — sandy soil leaches quickly. "
        "Split urea into 3 doses across 6 irrigation stages.",
        "wheat_sandy_low.txt",
    ),
    ("wheat", "clay", "medium"): (
        "Wheat on clay soil, medium rainfall: soil factor 1.5. "
        "Clay retains nutrients — apply full dose at sowing. "
        "Ensure drainage to prevent waterlogging.",
        "wheat_clay_medium.txt",
    ),
    ("rice", "clay", "high"): (
        "Rice on clay soil, high rainfall: base 100 kg/ha, soil factor 1.5. "
        "Maintain 5 cm standing water for 14 weeks. "
        "Transplant June–July. Drain 2 weeks before harvest.",
        "rice_clay_high.txt",
    ),
    ("maize", "loamy", "medium"): (
        "Maize on loamy soil, medium rainfall: base 90 kg/ha, soil factor 1.0. "
        "Apply basal NPK before planting. "
        "Critical irrigation at tasseling and silk emergence.",
        "maize_loamy_medium.txt",
    ),
    ("tomato", "loamy", "medium"): (
        "Tomato on loamy soil: base 150 kg/ha, intensive feeder. "
        "Drip 4–5 liters/plant/day at fruiting. "
        "Stake at 30 cm. Fruit set optimal 15–30°C. Apply in 4 splits.",
        "tomato_loamy_medium.txt",
    ),
}

# Plain KB sentences — no entity link
KB_SENTENCES: list[str] = [
    "Sandy soil factor 1.2: loses nitrogen quickly, apply 20% extra.",
    "Clay soil factor 1.5: heavy soil, apply 50% more fertilizer.",
    "Loamy soil factor 1.0: ideal balanced soil, standard rate.",
    "Wheat base 80 kg/ha. Rice 100 kg/ha. Maize 90 kg/ha.",
    "Low rainfall waterAdjustment 1.3: full irrigation required.",
    "Medium rainfall waterAdjustment 1.1: partial irrigation.",
    "High rainfall waterAdjustment 0.9: no irrigation needed.",
    "Fertilizer = basePerHectare × soilFactor × areaHectare × waterAdjustment.",
    "Estimated yield (tonnes) = areaHectare × 6 × soilFactor.",
    "Advisory score = total fertilizer + estimated yield.",
    "Urea is 46% nitrogen. Purchase kg = total N ÷ 0.46.",
    "Split nitrogen into doses to reduce leaching and volatilisation.",
]


def seed() -> None:
    eav   = EavStore()
    store = PgHybridStore()

    # collector: upsert each entity → {(crop,soil,rain): entity_id}
    entity_ids: dict[tuple, int] = {
```

```
        (crop, soil, rain): eav.upsert_entity(
            crop, soil, rain,
            base_per_hectare=base, soil_factor=sf, water_adjustment=wa,
        )
        for crop, soil, rain, base, sf, wa in ENTITY_SPECS
    }

    # set EAV attributes for each entity using set_attrs (one connection per entity)
    for key, eid in entity_ids.items():
        eav.set_attrs(eid, ENTITY_ATTRS[key])

    # insert content chunk and link to entity
    for key, eid in entity_ids.items():
        content, source = ENTITY_CONTENT[key]
        cid = store.insert(content=content, source=source)
        store.link_chunk_entity(cid, eid)

    # collector: insert all KB sentences (no entity link)
    kb_ids = [store.insert(content=s, source=KB_SOURCE) for s in KB_SENTENCES]

    print(f"Seeded {len(entity_ids)} entities, {len(kb_ids)} KB rows.")


if __name__ == "__main__":
    seed()
```

Phase 3 — Conversational RAG Assistant (Section 3)

```
# phase3_assistant/chatbot.py

import os
import google.generativeai as genai
from dataclasses import dataclass, field
from dotenv import load_dotenv
from phase2_storage.pg_hybrid_store import PgHybridStore

load_dotenv()

# ── Constants ──────────────────────────────────────────────────────────
GEMINI_API_KEY  = os.getenv("GEMINI_API_KEY")
GEN_MODEL       = "gemini-1.5-flash"
DEFAULT_TOP_K   = 4
DEFAULT_MAX_TURNS = 4
BULLET          = "• "
TURN_SEPARATOR  = "\n\n"

PROMPT_TEMPLATE = """\
You are AgriBot, an expert agricultural assistant.
{history}{rule_block}
Knowledge base (pgvector retrieval):
{context}

Farmer: {user_msg}
AgriBot:"""
```

```python
RULE_BLOCK_TEMPLATE = "\n[Rule Engine]:\n{result}\n"
HISTORY_TURN_TEMPLATE = "Farmer: {user}\nAgriBot: {assistant}\n\n"

genai.configure(api_key=GEMINI_API_KEY)


@dataclass
class ConversationalRagBot:
    store:     PgHybridStore = field(default_factory=PgHybridStore)
    history:   list[dict]    = field(default_factory=list)
    max_turns: int            = DEFAULT_MAX_TURNS

    def _retrieve(self, query: str, k: int = DEFAULT_TOP_K,
                  crop: str | None = None) -> list[str]:
        return self.store.retrieve_content_only(query, k=k, crop_filter=crop)

    def _build_context(self, chunks: list[str]) -> str:
        # collector: prefix every chunk with bullet character
        return "\n".join(BULLET + c for c in chunks)

    def _build_history(self) -> str:
        # collector: format last max_turns turns as history string
        return "".join(
            HISTORY_TURN_TEMPLATE.format(**turn)
            for turn in self.history[-self.max_turns:]
        )

    def chat(self, user_msg: str, rule_result: dict = None,
             crop_hint: str | None = None) -> str:
        chunks     = self._retrieve(user_msg, crop=crop_hint)
        rule_block = (
            RULE_BLOCK_TEMPLATE.format(result=rule_result)
            if rule_result else ""
        )
        prompt = PROMPT_TEMPLATE.format(
            history    = self._build_history(),
            rule_block = rule_block,
            context    = self._build_context(chunks),
            user_msg   = user_msg,
        )
        answer = genai.GenerativeModel(GEN_MODEL).generate_content(prompt).text
        self.history.append({"user": user_msg, "assistant": answer})
        return answer

    def reset(self) -> None:
        self.history.clear()
```

Phase 4 — RAG vs. Fine-Tuning (Section 4)

```python
# phase4_evaluation/decision.py

from dataclasses import dataclass
from enum import Enum, auto

# ── Constants ──────────────────────────────────────────────────────────
```

```
REASON_HYBRID    = ("LLM #1 parses → Lark rules compute → "
                    "pgvector RAG adds context → LLM #2 beautifies.")
REASON_RAG_LIVE  = "RAG with live DB for prices/weather."
REASON_RAG_STATIC= "pgvector top-k from static KB."
REASON_FINETUNE  = "Fine-tuning for report generation style."
REASON_RAG_FT    = "RAG + fine-tuning for grounded generation."


class Approach(Enum):
    RAG         = auto()
    RULE_ENGINE = auto()
    FINE_TUNE   = auto()
    HYBRID      = auto()


@dataclass
class QueryProfile:
    query: str
    requires_calculation: bool
    requires_latest_data: bool
    is_factual_lookup: bool
    is_style_generation: bool


def decide(p: QueryProfile) -> tuple[Approach, str]:
    # ordered decision table — constants for every reason string
    rules: list[tuple[bool, Approach, str]] = [
        (p.requires_calculation,
         Approach.HYBRID,     REASON_HYBRID),
        (p.requires_latest_data,
         Approach.RAG,        REASON_RAG_LIVE),
        (p.is_factual_lookup and not p.is_style_generation,
         Approach.RAG,        REASON_RAG_STATIC),
        (p.is_style_generation and not p.is_factual_lookup,
         Approach.FINE_TUNE,  REASON_FINETUNE),
    ]
    # collector: find first matching rule
    match = next(
        ((approach, reason) for cond, approach, reason in rules if cond),
        (Approach.HYBRID, REASON_RAG_FT),
    )
    return match
```

Phase 5 — IoT Docs Assistant (Section 5)

```
# phase5_iot_docs/agri_docs_assistant.py

import os, json
import google.generativeai as genai
from dotenv import load_dotenv
from phase2_storage.pg_hybrid_store import PgHybridStore

load_dotenv()

# ── Constants ──────────────────────────────────────────────────────────
```

```
GEMINI_API_KEY    = os.getenv("GEMINI_API_KEY")
GEN_MODEL         = "gemini-1.5-flash"
DEFAULT_TOP_K     = 4
SOURCE_FALLBACK   = "kb"
JSON_INDENT       = 2

PROMPT_TEMPLATE = """\
Agriculture IoT assistant. Give specific, actionable advice.

Documentation (pgvector):
{context}
{rule_block}{sensor_block}
Question: {question}
Answer:"""

SOURCE_LINE_TEMPLATE = "[{source}] {content}"
RULE_BLOCK_TEMPLATE   = "\nRule engine output:\n{data}\n"
SENSOR_BLOCK_TEMPLATE = "\nLive sensors:\n{data}\n"

genai.configure(api_key=GEMINI_API_KEY)


class AgriDocAssistant:
    def __init__(self):
        self.store = PgHybridStore()

    def _build_context(self, chunks: list[dict]) -> str:
        # collector: format each chunk as "[source] content"
        return "\n".join(
            SOURCE_LINE_TEMPLATE.format(
                source  = r.get("source", SOURCE_FALLBACK),
                content = r["content"],
            )
            for r in chunks
        )

    def answer(
        self,
        question:    str,
        sensor_data: dict | None = None,
        rule_output: dict | None = None,
        crop_hint:   str  | None = None,
        k:           int         = DEFAULT_TOP_K,
    ) -> str:
        chunks = self.store.retrieve_top_k(question, k=k, crop_filter=crop_hint)

        rule_block   = (
            RULE_BLOCK_TEMPLATE.format(
                data=json.dumps(rule_output, indent=JSON_INDENT))
            if rule_output else ""
        )
        sensor_block = (
            SENSOR_BLOCK_TEMPLATE.format(
                data=json.dumps(sensor_data, indent=JSON_INDENT))
            if sensor_data else ""
        )
        prompt = PROMPT_TEMPLATE.format(
            context      = self._build_context(chunks),
            rule_block   = rule_block,
            sensor_block = sensor_block,
            question     = question,
```

```
    )
    return genai.GenerativeModel(GEN_MODEL).generate_content(prompt).text
```

Phase 6 — Rule Engine + LLM #1 Parser + LLM #2 Beautifier

Step 6.1 — Memory

```
# rule_engine/memory.py

class Memory:
    """Thin fact store. Satisfies rule_matcher.py's two call sites exactly."""

    def __init__(self, initial_facts: dict = None):
        self._facts: dict = dict(initial_facts or {})

    def get(self, field: str):
        return self._facts.get(field)

    @property
    def facts(self) -> dict:
        return self._facts

    def set(self, field: str, value) -> None:
        self._facts[field] = value

    def snapshot(self) -> dict:
        return dict(self._facts)
```

Step 6.2 — Rule Engine

```
# rule_engine/engine.py

from pathlib import Path
from typing import Any
from .parser       import load_rules
from .rule_matcher import match_rule
from .memory       import Memory

# ── Constants ──────────────────────────────────────────────────────────
DSL_PATH = Path(__file__).parent / "fertilizer_rules.dsl"
ENCODING = "utf-8"

# Dependency order mirrors the DSL: soil/base/water before fertilizer,
# fertilizer before advisory.
RULE_EXECUTION_ORDER: tuple[str, ...] = (
    "soil_factor_sandy",
    "soil_factor_clay",
    "soil_factor_loamy",
    "wheat_base_fertilizer",
    "rice_base_fertilizer",
    "maize_base_fertilizer",
    "rainfall_low_adjustment",
```

```python
    "rainfall_medium_adjustment",
    "rainfall_high_adjustment",
    "fertilizer_calculation",
    "irrigation_requirement",
    "irrigation_requirement_medium",
    "irrigation_requirement_high",
    "yield_estimation",
    "advisory_message",
)

# AST node type → operation (for expression evaluator)
NODE_OPS: dict[str, str] = {
    "add": "+", "sub": "-", "mul": "*", "div": "/",
}


class RuleEngine:
    def __init__(self, dsl_path: Path = DSL_PATH):
        dsl_text  = dsl_path.read_text(encoding=ENCODING)
        all_rules = load_rules(dsl_text)
        # collector: build name → rule dict
        self._rules: dict[str, Any] = {r.name: r for r in all_rules}

    def run(self, facts: dict) -> dict:
        """
        Fire all rules in dependency order.
        Collector style: filter to matching rules → evaluate → write to memory.
        """
        memory = Memory(facts)
        # for each rule name in order: look up, match, evaluate, set
        for rule_name in RULE_EXECUTION_ORDER:
            rule = self._rules.get(rule_name)
            if rule and match_rule(rule, memory, []):
                assignment = rule.then_clause.assignment
                memory.set(assignment.name, self._eval(assignment.expr, memory))
        return memory.snapshot()

    def _eval(self, node: Any, memory: Memory):
        t = getattr(node, "data", None)
        if t == "number":   return float(node.children[0])
        if t == "string":   return str(node.children[0]).strip('"')
        if t == "var":
            v = memory.get(str(node.children[0]))
            if v is None:
                raise ValueError(f"Undefined variable: {node.children[0]}")
            return v
        if t in ("add", "sub", "mul"):
            lhs = self._eval(node.children[0], memory)
            rhs = self._eval(node.children[1], memory)
            return lhs + rhs if t == "add" else lhs - rhs if t == "sub" else lhs * rhs
        if t == "div":
            rhs = self._eval(node.children[1], memory)
            if rhs == 0:
                raise ZeroDivisionError("Division by zero in rule")
            return self._eval(node.children[0], memory) / rhs
        if t == "min_func":
            return min(self._eval(node.children[0], memory),
                       self._eval(node.children[1], memory))
        if t == "max_func":
            return max(self._eval(node.children[0], memory),
                       self._eval(node.children[1], memory))
```

```
        try:    return float(node)
        except: return str(node).strip('"')
```

Step 6.3 — LLM #1: Query Parser

```
# llm/query_parser.py

import os, re, json
import google.generativeai as genai
from dotenv import load_dotenv

load_dotenv()

# ── Constants ──────────────────────────────────────────────────────────
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
GEN_MODEL      = "gemini-1.5-flash"

VALID_CROPS:    tuple[str, ...] = ("wheat", "rice", "maize")
VALID_SOILS:    tuple[str, ...] = ("sandy", "clay", "loamy")
VALID_RAINFALL: tuple[str, ...] = ("low", "medium", "high")

# Field validation spec: (field_name, valid_values, label_for_error)
FIELD_SPECS: tuple[tuple, ...] = (
    ("crop",     VALID_CROPS,    "wheat, rice, or maize"),
    ("soilType", VALID_SOILS,    "sandy, clay, or loamy"),
    ("rainfall", VALID_RAINFALL, "low, medium, or high"),
)

AREA_FIELD   = "areaHectare"
FENCE_REGEX  = r"^```[a-z]*\n?"
FENCE_STRIP  = "` \n"

PARSER_SYSTEM = """\
You are an agricultural query parser.
Extract EXACTLY these four fields:
  crop        → one of: "wheat", "rice", "maize"
  soilType    → one of: "sandy", "clay", "loamy"
  areaHectare → positive number (hectares)
  rainfall    → one of: "low", "medium", "high"
Set missing fields to null. Output ONLY valid JSON, no markdown.\
"""

EXTRACT_PROMPT = 'Extract from: "{query}"'
ERR_NON_JSON   = "LLM #1 returned non-JSON: {raw}\n{exc}"
ERR_MISSING    = "'{field}' is missing. Options: {label}."
ERR_INVALID    = "'{field}' value '{val}' not supported. Options: {label}."
ERR_AREA_MISS  = "'areaHectare' is missing. State the field size in hectares."
ERR_AREA_ZERO  = "'areaHectare' must be > 0."
ERR_AREA_TYPE  = "'areaHectare' must be a number. Got: {val}"
ERR_SUMMARY    = (
    "Missing or invalid fields:\n{issues}\n\n"
    'Please rephrase. Example: "I have 2 ha of wheat on sandy soil, low rainfall."'
)

genai.configure(api_key=GEMINI_API_KEY)


class QueryParser:
```

```python
    def __init__(self):
        self.model = genai.GenerativeModel(
            GEN_MODEL, system_instruction=PARSER_SYSTEM
        )

    def parse(self, query: str) -> dict:
        raw  = self.model.generate_content(
            EXTRACT_PROMPT.format(query=query)
        ).text.strip()
        raw  = re.sub(FENCE_REGEX, "", raw).rstrip(FENCE_STRIP)
        try:
            parsed = json.loads(raw)
        except json.JSONDecodeError as exc:
            raise ValueError(ERR_NON_JSON.format(raw=raw, exc=exc))
        return self._validate(parsed)

    def _validate(self, parsed: dict) -> dict:
        # collector: gather all error messages from field checks
        errors = [
            *self._check_enum_fields(parsed),
            *self._check_area(parsed),
        ]
        if errors:
            raise ValueError(
                ERR_SUMMARY.format(issues="\n".join(f"  • {e}" for e in errors))
            )
        return parsed

    def _check_enum_fields(self, parsed: dict) -> list[str]:
        """Collector: validate each enum field, return list of error strings."""
        errors = []
        for field, valid, label in FIELD_SPECS:
            val = (parsed.get(field) or "").lower().strip()
            if not val:
                errors.append(ERR_MISSING.format(field=field, label=label))
            elif val not in valid:
                errors.append(ERR_INVALID.format(field=field, val=val, label=label))
            else:
                parsed[field] = val
        return errors

    def _check_area(self, parsed: dict) -> list[str]:
        """Validate areaHectare, return list of error strings (0 or 1 items)."""
        area = parsed.get(AREA_FIELD)
        if area is None:
            return [ERR_AREA_MISS]
        try:
            area = float(area)
            if area <= 0:
                return [ERR_AREA_ZERO]
            parsed[AREA_FIELD] = area
            return []
        except (TypeError, ValueError):
            return [ERR_AREA_TYPE.format(val=area)]
```

Step 6.4 — LLM #2: Answer Beautifier

```python
# llm/answer_beautifier.py

import os
import google.generativeai as genai
from dotenv import load_dotenv

load_dotenv()

# ── Constants ──────────────────────────────────────────────────────────
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
GEN_MODEL      = "gemini-1.5-flash"

# Human-readable labels for rule engine output keys
COMPUTED_LABELS: dict[str, str] = {
    "soilFactor":         "Soil adjustment factor",
    "basePerHectare":     "Base fertilizer rate (kg/ha)",
    "waterAdjustment":    "Water/rainfall adjustment factor",
    "fertilizer":         "Total fertilizer required (kg)",
    "irrigationRequired": "Irrigation required (0=none, 1=full)",
    "estimatedYield":     "Estimated yield (tonnes)",
    "advisoryScore":      "Advisory score (fertilizer + yield)",
}

BEAUTIFIER_SYSTEM = """\
You are AgriBot, a friendly expert agricultural advisor.
You receive:
  1. The farmer's question
  2. Exact computed values from a rule engine (do NOT change these numbers)
  3. Supporting context from a knowledge base (pgvector retrieval)
  4. Crop profile attributes from the EAV store (variable per crop)

Format your answer with clear sections:
  Summary | Fertilizer Plan | Irrigation | Yield Estimate |
  Crop Profile Notes | Recommendations
Include units (kg, ha, tonnes). State clearly when irrigationRequired = 0.0.\
"""

PROMPT_TEMPLATE = """\
Farmer's question: "{query}"

Rule Engine Output (exact — do not modify):
{rule_block}
{rag_block}{eav_block}
Write a clear, structured agronomic response.\
"""

INPUT_SECTION_HEADER   = "Input facts:"
COMPUTED_SECTION_HEADER= "Computed values:"
INPUT_LINE             = "  {key}: {val}"
COMPUTED_LINE          = "  {label}: {val}"
KB_HEADER              = "\nKnowledge Base:\n"
KB_BULLET              = "  • {chunk}"
EAV_HEADER             = "\nCrop Profile (EAV attributes):\n"
EAV_LINE               = "  {key}: {val}"
FLOAT_DECIMALS         = 2

genai.configure(api_key=GEMINI_API_KEY)


class AnswerBeautifier:
```

```python
    def __init__(self):
        self.model = genai.GenerativeModel(
            GEN_MODEL, system_instruction=BEAUTIFIER_SYSTEM
        )

    def beautify(
        self,
        original_query: str,
        rule_output:    dict,
        parsed_input:   dict,
        rag_context:    list[str] | None = None,
        crop_profile:   dict     | None  = None,
    ) -> str:
        prompt = PROMPT_TEMPLATE.format(
            query      = original_query,
            rule_block = self._format_rule(rule_output, parsed_input),
            rag_block  = self._format_rag(rag_context),
            eav_block  = self._format_eav(crop_profile),
        )
        return self.model.generate_content(prompt).text

    def _format_rule(self, rule_output: dict, parsed_input: dict) -> str:
        """Collector: build input section + computed section as joined strings."""
        input_keys = set(parsed_input.keys())

        input_lines = [INPUT_SECTION_HEADER] + [
            INPUT_LINE.format(key=k, val=v)
            for k, v in parsed_input.items()
        ]
        computed_lines = [COMPUTED_SECTION_HEADER] + [
            COMPUTED_LINE.format(
                label = COMPUTED_LABELS.get(k, k),
                val   = round(v, FLOAT_DECIMALS) if isinstance(v, float) else v,
            )
            for k, v in rule_output.items()
            if k not in input_keys
        ]
        return "\n".join(input_lines + computed_lines)

    def _format_rag(self, rag_context: list[str] | None) -> str:
        if not rag_context:
            return ""
        # collector: bullet every KB chunk
        lines = [KB_HEADER] + [KB_BULLET.format(chunk=c) for c in rag_context]
        return "\n".join(lines)

    def _format_eav(self, crop_profile: dict | None) -> str:
        if not crop_profile:
            return ""
        # collector: format every EAV attribute as a key-value line
        lines = [EAV_HEADER] + [
            EAV_LINE.format(key=k, val=v)
            for k, v in crop_profile.items()
        ]
        return "\n".join(lines)
```

Step 6.5 — Full Hybrid Pipeline

```python
# pipeline/hybrid_pipeline.py

import os
from pathlib import Path
import google.generativeai as genai
from dotenv import load_dotenv

from rule_engine.engine          import RuleEngine
from llm.query_parser            import QueryParser
from llm.answer_beautifier       import AnswerBeautifier
from phase2_storage.pg_hybrid_store import PgHybridStore

load_dotenv()

# ── Constants ──────────────────────────────────────────────────────────
GEMINI_API_KEY    = os.getenv("GEMINI_API_KEY")
DEFAULT_TOP_K     = 4
DSL_RELATIVE_PATH = Path("rule_engine") / "fertilizer_rules.dsl"

CLARIFY_PREFIX    = "I need a few more details.\n\n"
RESULT_KEYS       = ("answer", "parsed_input", "rule_output",
                     "rag_chunks", "crop_profile", "error")

HISTORY_FARMER    = "farmer"
HISTORY_AGRIBOT   = "agribot"

genai.configure(api_key=GEMINI_API_KEY)


class HybridPipeline:
    """
    Full pipeline: LLM #1 → Lark rules → crop-scoped pgvector → EAV → LLM #2.
    Constants-first. Collector style throughout.
    """

    def __init__(self, dsl_path: Path = None):
        dsl = dsl_path or (Path(__file__).parent.parent / DSL_RELATIVE_PATH)
        self.parser     = QueryParser()
        self.engine     = RuleEngine(dsl_path=dsl)
        self.beautifier = AnswerBeautifier()
        self.store      = PgHybridStore()
        self.history:   list[dict] = []

    def _retrieve(self, query: str, k: int = DEFAULT_TOP_K,
                  crop: str | None = None) -> list[str]:
        return self.store.retrieve_content_only(query, k=k, crop_filter=crop)

    def _empty_result(self) -> dict:
        # collector: build result skeleton from constant key tuple
        return dict.fromkeys(RESULT_KEYS, None) | {"rag_chunks": [], "answer": "", "error": None}

    def chat(self, user_query: str) -> dict:
        result = self._empty_result()

        # Step 0: broad retrieval before crop is known
        result["rag_chunks"] = self._retrieve(user_query)

        # Step 1: LLM #1 — NL → validated JSON
        try:
            parsed = self.parser.parse(user_query)
```

```
            result["parsed_input"] = parsed
        except ValueError as exc:
            result["answer"] = CLARIFY_PREFIX + str(exc)
            result["error"]  = str(exc)
            self._log(user_query, result["answer"])
            return result

        # Step 1b: crop-scoped pgvector re-retrieval
        crop = parsed.get("crop")
        if crop:
            result["rag_chunks"] = self._retrieve(user_query, crop=crop)

        # Step 2: rule engine — deterministic calculation
        rule_output          = self.engine.run(parsed)
        result["rule_output"] = rule_output

        # Step 3: EAV crop profile
        entity = self.store.get_rule_facts(
            parsed["crop"], parsed["soilType"], parsed["rainfall"]
        )
        if entity:
            result["crop_profile"] = self.store.get_crop_profile(entity["id"])

        # Step 4: LLM #2 — beautify
        result["answer"] = self.beautifier.beautify(
            original_query = user_query,
            rule_output    = rule_output,
            parsed_input   = parsed,
            rag_context    = result["rag_chunks"],
            crop_profile   = result["crop_profile"],
        )
        self._log(user_query, result["answer"])
        return result

    def _log(self, query: str, answer: str) -> None:
        self.history.extend([
            {"role": HISTORY_FARMER,  "text": query},
            {"role": HISTORY_AGRIBOT, "text": answer},
        ])

    def reset_history(self) -> None:
        self.history.clear()
```

Tests

```
# tests/test_eav_store.py

import pytest
from unittest.mock import patch, MagicMock
from phase2_storage.eav_store import (
    EavStore, _infer_type, _value_col, _read_value,
    VALUE_COL, TYPE_MAP,
)

# ── Constants used in tests ──────────────────────────────────────────
```

```
ENTITY_ID       = 1
CROP_WHEAT      = "wheat"
SOIL_SANDY      = "sandy"
RAIN_LOW        = "low"
BASE_RATE       = 80.0
SOIL_FACTOR     = 1.2
WATER_ADJ       = 1.3
MOCK_ENTITY_ID  = 42
ATTR_GROWTH     = "growth_days"
ATTR_PH         = "pH_range_min"
ATTR_PRUNE      = "pruning_required"
ATTR_LEACH      = "leaching_risk"
VAL_GROWTH      = 120
VAL_PH          = 6.0
VAL_LEACH       = "high"
VAL_PRUNE_BOOL  = True
ATTR_RELATED_1  = "related_crop_1"
ATTR_RELATED_2  = "related_crop_2"
VAL_BARLEY      = "barley"
VAL_OATS        = "oats"


def _mock_conn(fetchone=None, fetchall=None):
    cur  = MagicMock()
    cur.__enter__  = MagicMock(return_value=cur)
    cur.__exit__   = MagicMock(return_value=False)
    cur.fetchone.return_value  = fetchone
    cur.fetchall.return_value  = fetchall or []
    conn = MagicMock()
    conn.__enter__ = MagicMock(return_value=conn)
    conn.__exit__  = MagicMock(return_value=False)
    conn.cursor.return_value = cur
    return conn, cur


class TestHelpers:
    # collector: parametrize all type inference cases
    TYPE_CASES: list[tuple] = [
        (True,  "boolean"),
        (5,     "integer"),
        (3.14,  "float"),
        ("hi",  "text"),
    ]

    @pytest.mark.parametrize("value,expected", TYPE_CASES)
    def test_infer_type(self, value, expected):
        assert _infer_type(value) == expected

    # collector: parametrize all value_col mapping cases
    COL_CASES: list[tuple] = list(VALUE_COL.items())

    @pytest.mark.parametrize("vtype,expected_col", COL_CASES)
    def test_value_col_mapping(self, vtype, expected_col):
        assert _value_col(vtype) == expected_col

    def test_read_value_float(self):
        row = {"value_type": "float", "value_integer": None,
               "value_float": VAL_PH, "value_text": None, "value_boolean": None}
        assert _read_value(row) == VAL_PH

    def test_read_value_boolean(self):
```

```
        row = {"value_type": "boolean", "value_integer": None,
               "value_float": None, "value_text": None,
               "value_boolean": VAL_PRUNE_BOOL}
        assert _read_value(row) is True


class TestEavStore:

    def test_upsert_entity_returns_id(self):
        conn, _ = _mock_conn(fetchone=(MOCK_ENTITY_ID,))
        with patch("phase2_storage.eav_store.psycopg2.connect", return_value=conn):
            eid = EavStore().upsert_entity(
                CROP_WHEAT, SOIL_SANDY, RAIN_LOW,
                BASE_RATE, SOIL_FACTOR, WATER_ADJ,
            )
        assert eid == MOCK_ENTITY_ID

    # collector: parametrize set_attr column routing
    SET_ATTR_CASES: list[tuple] = [
        (ATTR_GROWTH, VAL_GROWTH, "value_integer"),
        (ATTR_PH,     VAL_PH,     "value_float"),
        (ATTR_PRUNE,  True,       "value_boolean"),
        (ATTR_LEACH,  VAL_LEACH,  "value_text"),
    ]

    @pytest.mark.parametrize("attr,val,expected_col", SET_ATTR_CASES)
    def test_set_attr_column_routing(self, attr, val, expected_col):
        conn, cur = _mock_conn()
        captured  = []
        cur.execute = lambda sql, params: captured.append(sql)
        with patch("phase2_storage.eav_store.psycopg2.connect", return_value=conn):
            EavStore().set_attr(ENTITY_ID, attr, val)
        assert any(expected_col in s for s in captured)

    def test_set_attr_is_idempotent(self):
        conn, cur = _mock_conn()
        captured  = []
        cur.execute = lambda sql, params: captured.append(sql)
        with patch("phase2_storage.eav_store.psycopg2.connect", return_value=conn):
            EavStore().set_attr(ENTITY_ID, ATTR_GROWTH, VAL_GROWTH)
        assert any("ON CONFLICT" in s for s in captured)

    def test_get_attr_typed_value(self):
        row = {"value_type": "integer", "value_integer": VAL_GROWTH,
               "value_float": None, "value_text": None, "value_boolean": None}
        conn, _ = _mock_conn(fetchone=row)
        with patch("phase2_storage.eav_store.psycopg2.connect", return_value=conn):
            val = EavStore().get_attr(ENTITY_ID, ATTR_GROWTH)
        assert val == VAL_GROWTH

    def test_get_attr_returns_none_for_missing(self):
        conn, _ = _mock_conn(fetchone=None)
        with patch("phase2_storage.eav_store.psycopg2.connect", return_value=conn):
            assert EavStore().get_attr(ENTITY_ID, "water_depth_cm") is None

    def test_get_attrs_returns_flat_dict(self):
        rows = [
            {"attribute": ATTR_GROWTH, "value_type": "integer",
             "value_integer": VAL_GROWTH, "value_float": None,
             "value_text": None, "value_boolean": None},
            {"attribute": ATTR_LEACH, "value_type": "text",
```

```
             "value_integer": None, "value_float": None,
             "value_text": VAL_LEACH, "value_boolean": None},
        ]
        conn, _ = _mock_conn(fetchall=rows)
        with patch("phase2_storage.eav_store.psycopg2.connect", return_value=conn):
            attrs = EavStore().get_attrs(ENTITY_ID)
        assert attrs == {ATTR_GROWTH: VAL_GROWTH, ATTR_LEACH: VAL_LEACH}

    def test_related_crop_attrs_startswith_prefix(self):
        rows = [
            {"attribute": ATTR_RELATED_1, "value_type": "text",
             "value_integer": None, "value_float": None,
             "value_text": VAL_BARLEY, "value_boolean": None},
            {"attribute": ATTR_RELATED_2, "value_type": "text",
             "value_integer": None, "value_float": None,
             "value_text": VAL_OATS, "value_boolean": None},
        ]
        conn, _ = _mock_conn(fetchall=rows)
        with patch("phase2_storage.eav_store.psycopg2.connect", return_value=conn):
            attrs = EavStore().get_attrs_by_name(CROP_WHEAT)
        # collector: filter by prefix
        related = [v for k, v in attrs.items() if k.startswith("related_crop_")]
        assert set(related) == {VAL_BARLEY, VAL_OATS}


# tests/test_pg_store.py

import pytest
from unittest.mock import patch, MagicMock
from phase2_storage.pg_hybrid_store import PgHybridStore

# ── Constants ──────────────────────────────────────────────────────────
MOCK_VEC         = [0.1] * 768
MOCK_SIM         = 0.87
QUERY_WHEAT      = "wheat fertilizer"
QUERY_RICE       = "rice irrigation"
CROP_RICE        = "rice"
CROP_WHEAT       = "wheat"
K_3              = 3
K_4              = 4
K_1              = 1
TABLE_ENTITIES   = "crop_entities"
OP_COSINE        = "<=>"
COMPANION_QUERY  = "companion crop advice"


def _mock_store(fetchall=None, fetchone=None):
    cur  = MagicMock()
    cur.__enter__  = MagicMock(return_value=cur)
    cur.__exit__   = MagicMock(return_value=False)
    cur.fetchall.return_value = fetchall or []
    cur.fetchone.return_value = fetchone
    conn = MagicMock()
    conn.__enter__ = MagicMock(return_value=conn)
    conn.__exit__  = MagicMock(return_value=False)
    conn.cursor.return_value = cur
    return conn, cur


class TestPgHybridStore:
```

```python
    def _run_retrieve(self, crop_filter=None, k=K_4, fetchall=None):
        """Helper: run retrieve_top_k with mocked connection, return captured SQL."""
        captured = []
        conn, cur = _mock_store(fetchall=fetchall or [])
        cur.execute = lambda sql, p: captured.append(sql)
        with patch("phase2_storage.pg_hybrid_store.psycopg2.connect",
                   return_value=conn), \
             patch("phase2_storage.pg_hybrid_store.register_vector"), \
             patch("phase2_storage.pg_hybrid_store.genai.embed_content",
                   return_value={"embedding": MOCK_VEC}):
            PgHybridStore().retrieve_top_k(QUERY_WHEAT, k=k,
                                           crop_filter=crop_filter)
        return captured

    def test_retrieval_uses_cosine_operator(self):
        captured = self._run_retrieve()
        assert any(OP_COSINE in s for s in captured)

    def test_crop_filter_joins_entity_table(self):
        captured = self._run_retrieve(crop_filter=CROP_RICE)
        assert any(TABLE_ENTITIES in s for s in captured)

    def test_no_crop_filter_no_entity_join(self):
        captured = self._run_retrieve()
        assert all(TABLE_ENTITIES not in s for s in captured)

    def test_cosine_similarity_returned_in_rows(self):
        rows = [{"content": "wheat text", "cosine_similarity": MOCK_SIM,
                 "source": "wheat.txt"}]
        conn, cur = _mock_store(fetchall=rows)
        with patch("phase2_storage.pg_hybrid_store.psycopg2.connect",
                   return_value=conn), \
             patch("phase2_storage.pg_hybrid_store.register_vector"), \
             patch("phase2_storage.pg_hybrid_store.genai.embed_content",
                   return_value={"embedding": MOCK_VEC}):
            results = PgHybridStore().retrieve_top_k(QUERY_WHEAT, k=K_1)
        assert results[0]["cosine_similarity"] == pytest.approx(MOCK_SIM)

    def test_traverse_related_reads_eav_then_pgvector(self):
        """traverse_related: EAV attr read first, then <=> vector query."""
        eav_mock = MagicMock()
        eav_mock.get_attrs_by_name.return_value = {
            "related_crop_1": "barley",
            "related_crop_2": "oats",
            "growth_days":    VAL_GROWTH := 120,
        }
        captured = []
        conn, cur = _mock_store()
        cur.execute = lambda sql, p: captured.append(sql)
        with patch("phase2_storage.pg_hybrid_store.psycopg2.connect",
                   return_value=conn), \
             patch("phase2_storage.pg_hybrid_store.register_vector"), \
             patch("phase2_storage.pg_hybrid_store.genai.embed_content",
                   return_value={"embedding": MOCK_VEC}):
            store     = PgHybridStore()
            store.eav = eav_mock
            store.traverse_related(CROP_WHEAT, COMPANION_QUERY, k=K_3)
        assert any(OP_COSINE in s for s in captured)


# tests/test_rule_engine.py
```

```python
import pytest
from rule_engine.engine import RuleEngine, RULE_EXECUTION_ORDER
from pathlib import Path

# ── Constants ──────────────────────────────────────────────────────────────
DSL_PATH        = Path("rule_engine/fertilizer_rules.dsl")
CROP_WHEAT      = "wheat"
CROP_RICE       = "rice"
CROP_MAIZE      = "maize"
SOIL_SANDY      = "sandy"
SOIL_CLAY       = "clay"
SOIL_LOAMY      = "loamy"
RAIN_LOW        = "low"
RAIN_HIGH       = "high"
RAIN_MEDIUM     = "medium"
FACTOR_SANDY    = 1.2
FACTOR_CLAY     = 1.5
FACTOR_LOAMY    = 1.0
# fertilizer = base × soil_factor × area × water_adj
# wheat/sandy/low/2ha = 80 × 1.2 × 2.0 × 1.3 = 249.6
EXPECTED_FERTILIZER_W_S_L_2 = 249.6
# yield = area × 6 × soil_factor  (wheat/loamy/2ha = 2.0×6×1.0 = 12.0)
EXPECTED_YIELD_W_LO_2       = 12.0

REQUIRED_OUTPUT_KEYS: tuple[str, ...] = (
    "soilFactor", "basePerHectare", "waterAdjustment",
    "fertilizer", "irrigationRequired", "estimatedYield", "advisoryScore",
)


@pytest.fixture
def engine():
    return RuleEngine(dsl_path=DSL_PATH)


def _facts(crop=CROP_WHEAT, soil=SOIL_SANDY,
           area=1.0, rain=RAIN_LOW) -> dict:
    return {"crop": crop, "soilType": soil,
            "areaHectare": area, "rainfall": rain}


class TestRuleEngine:

    # collector: parametrize soil factor cases
    SOIL_FACTOR_CASES: list[tuple] = [
        (CROP_WHEAT, SOIL_SANDY, RAIN_LOW,    FACTOR_SANDY),
        (CROP_WHEAT, SOIL_CLAY,  RAIN_MEDIUM, FACTOR_CLAY),
        (CROP_RICE,  SOIL_LOAMY, RAIN_HIGH,   FACTOR_LOAMY),
    ]

    @pytest.mark.parametrize("crop,soil,rain,expected", SOIL_FACTOR_CASES)
    def test_soil_factor(self, engine, crop, soil, rain, expected):
        out = engine.run(_facts(crop, soil, 1.0, rain))
        assert out["soilFactor"] == pytest.approx(expected)

    def test_fertilizer_wheat_sandy_low_2ha(self, engine):
        out = engine.run(_facts(CROP_WHEAT, SOIL_SANDY, 2.0, RAIN_LOW))
        assert out["fertilizer"] == pytest.approx(
            EXPECTED_FERTILIZER_W_S_L_2, rel=1e-3
        )
```

```python
    def test_irrigation_zero_on_high_rainfall(self, engine):
        out = engine.run(_facts(CROP_RICE, SOIL_CLAY, 1.0, RAIN_HIGH))
        assert out["irrigationRequired"] == pytest.approx(0.0)

    def test_advisory_score_equals_fertilizer_plus_yield(self, engine):
        out = engine.run(_facts(CROP_MAIZE, SOIL_LOAMY, 2.0, RAIN_MEDIUM))
        assert out["advisoryScore"] == pytest.approx(
            out["fertilizer"] + out["estimatedYield"], rel=1e-3
        )

    def test_all_output_keys_present(self, engine):
        out = engine.run(_facts(CROP_WHEAT, SOIL_CLAY, 1.0, RAIN_MEDIUM))
        # collector: check all required keys in one assertion
        missing = [k for k in REQUIRED_OUTPUT_KEYS if k not in out]
        assert missing == [], f"Missing keys: {missing}"

    def test_rule_execution_order_is_complete(self):
        """All rules in the DSL must appear in RULE_EXECUTION_ORDER."""
        engine    = RuleEngine(dsl_path=DSL_PATH)
        dsl_names = set(engine._rules.keys())
        order_set = set(RULE_EXECUTION_ORDER)
        # collector: find DSL rules not in the order tuple
        missing   = [r for r in dsl_names if r not in order_set]
        assert missing == [], f"Rules missing from order: {missing}"
```

Deployment Checklist

```
Coding conventions
 ✓ Every model name, table name, SQL string, prompt template
   is a module-level constant — no literals inside functions
 ✓ Every loop that builds a list/dict is a comprehension or
   generator expression (no for-append accumulation)
 ✓ Validation errors collected with list comprehension, joined at end
 ✓ Decision rules in Phase 4 use next() over generator, not if-chain
 ✓ Seed data driven entirely by constant dicts (ENTITY_SPECS,
   ENTITY_ATTRS, ENTITY_CONTENT, KB_SENTENCES)

EAV Schema
 ✓ crop_entities UNIQUE(crop_name, soil_type, rainfall)
 ✓ crop_attributes value_type CHECK constraint
 ✓ set_attrs() uses one connection for bulk attribute upserts
 ✓ IVFFlat index on embedding (lists=100)
 ✓ B-tree index on (crop_name, soil_type) and (entity_id, attribute)

pgvector Retrieval
 ✓ SQL templates are module-level constants (SQL_TOPK_ALL, SQL_TOPK_CROP)
 ✓ SQL template selected with ternary, not if-block
 ✓ query_vec bound twice per query (SELECT similarity + ORDER BY)
 ✓ crop_filter join goes through chunk_entity_map → crop_entities
 ✓ cosine_similarity = 1 - (embedding <=> query_vec)

Pipeline
 ✓ _empty_result() uses dict.fromkeys(RESULT_KEYS) — constants-first
 ✓ history.extend([...]) — single call, not two appends
```

```
 ✓ Crop-scoped re-retrieval after LLM #1 parses crop field

Tests
 ✓ All test literals (crop names, values, SQL fragments) are constants
 ✓ Parametrize decorators replace repeated test methods (collector style)
 ✓ _infer_type cases parametrized from TYPE_CASES list
 ✓ set_attr column routing cases parametrized from SET_ATTR_CASES list
 ✓ Soil factor cases parametrized from SOIL_FACTOR_CASES list
 ✓ Missing-key check uses list comprehension, not for-loop
```

Quick Start

```
python3.13 -m venv agri_env && source agri_env/bin/activate
pip install -r requirements.txt
export GEMINI_API_KEY="your_key"
export POSTGRES_DSN="postgresql://user:pass@localhost/agridb"

psql $POSTGRES_DSN -f phase2_storage/schema.sql
python phase2_storage/seed_data.py

# smoke test constants and collector style
python -c "
from phase2_storage.eav_store import EavStore, VALUE_COL, TYPE_MAP
s   = EavStore()
eid = s.upsert_entity('wheat','sandy','low', 80, 1.2, 1.3)
s.set_attrs(eid, {
    'growth_days':    (120,    'integer'),
    'leaching_risk':  ('high', 'text'),
    'pruning_required': (False,'boolean'),
})
print(s.get_attrs(eid))
print('VALUE_COL:', VALUE_COL)
print('TYPE_MAP:', TYPE_MAP)
"

python main.py
pytest tests/ -v
```

Python 3.13 · Gemini API · Lark parser · pgvector · EAV All literals are constants · All accumulators are comprehensions